

Game 1: AI Maze Escape using BFS

Code:

```
from collections import deque
maze = [
['S', '#', ''],
['#', ' ', ''],
[' ', '#', ''],
[' ', '#', 'G']
]
rows=len(maze)
cols=len(maze[0])
moves=[(1,0),(-1,0),(0,1),(0,-1)]
def bfs():
    queue=deque()
    queue.append((0,0,[]))
    visited=set()
    while queue:
        x,y,path=queue.popleft()
        if (x,y) in visited:
            continue
        visited.add((x,y))
        path=path+[(x,y)]
        if maze[x][y]=='G':
            return path
        for dx,dy in moves:
            nx=x+dx
            ny=y+dy
            if 0<=nx<rows and 0<=ny<cols:
                if maze[nx][ny]!='#':
                    queue.append((nx,ny,path))
    path=bfs()
    print("Shortest Path:")
    print(path)
    print("Number of Steps:",len(path)-1)
```

OUTPUT:

```
1 from collections import deque
2
3 maze = [
4 ['S', ' ', '#', ' ', ' '],
5 ['#', ' ', ' ', ' ', ' '],
6 [' ', ' ', '#', ' ', ' '],
7 [' ', '#', ' ', ' ', 'G']
8 ]
9
10 rows=len(maze)
11 cols=len(maze[0])
12
13 moves=[(1,0),(-1,0),(0,1),(0,-1)]
14
15 def bfs():
16
17     queue=deque()
18     queue.append((0,0,[]))
19
20     visited=set()
21
22     while queue:
23
24         x,y,path=queue.popleft()
25
26         if (x,y) in visited:
```

```
Shortest Path:
(0, 0), (0, 1), (1, 1), (1, 2), (1, 3), (2, 3), (3, 3)]
Number of Steps: 6
```

```
..Program finished with exit code 0
Press ENTER to exit console.
```

GAME 2:N-Queens Problem (12 Queens)

CODE:N=12

board=[-1]*N

def safe(row,col):

for i in range(row):

if board[i]==col:

return False

if abs(board[i]-col)==abs(i-row):

return False

return True

def solve(row):

if row==N:

return True

for col in range(N):

if safe(row,col):

board[row]=col

if solve(row+1):

return True

return False

solve(0)

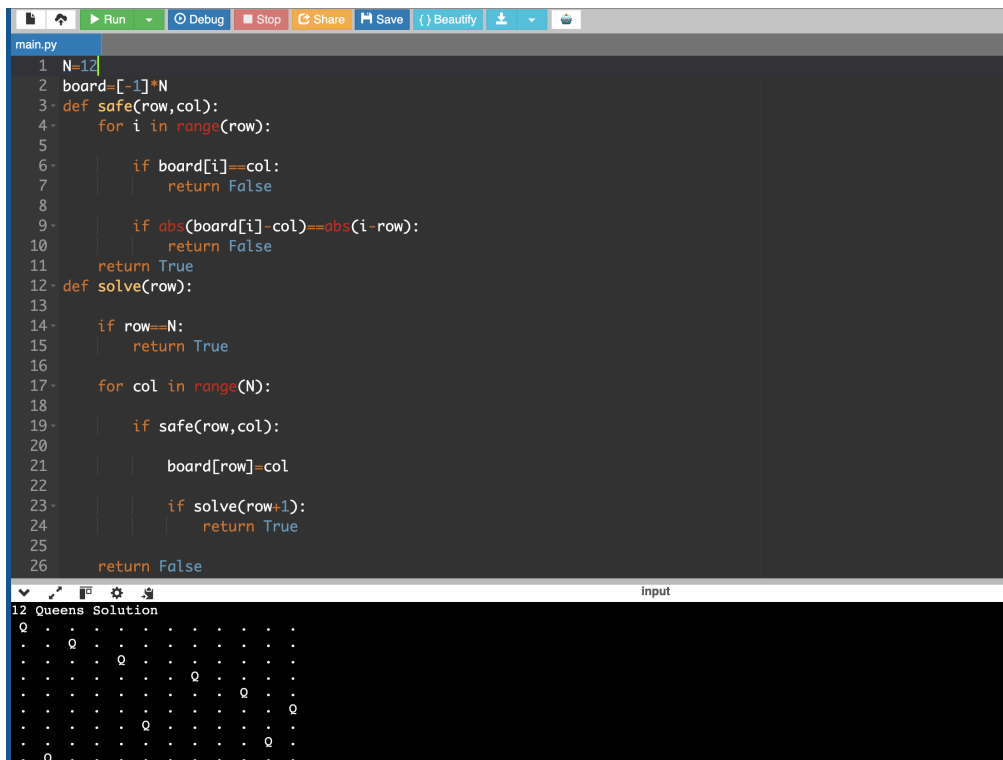
print("12 Queens Solution")

```

for r in range(N):
    line=""
    for c in range(N):
        if board[r]==c:
            line+=" Q "
        else:
            line+=" . "
    print(line)

```

OUTPUT:



The screenshot shows a code editor with a Python script for the N-Queens problem. The script defines a `safe` function to check if a queen can be placed at a given row and column, and a `solve` function to find all solutions. The output window shows the solutions for N=12, displaying a grid where 'Q' represents a queen and '.' represents an empty space.

```

main.py
1 N=12
2 board=[-1]*N
3 def safe(row,col):
4     for i in range(row):
5
6         if board[i]==col:
7             return False
8
9         if abs(board[i]-col)==abs(i-row):
10            return False
11    return True
12 def solve(row):
13
14    if row==N:
15        return True
16
17    for col in range(N):
18
19        if safe(row,col):
20
21            board[row]=col
22
23            if solve(row+1):
24                return True
25
26    return False

```

12 Queens Solution

```

Q . . . . . . . . . . . .
. Q . . . . . . . . . .
. . Q . . . . . . . . .
. . . Q . . . . . . . .
. . . . Q . . . . . . .
. . . . . Q . . . . . .
. . . . . . Q . . . . .
. . . . . . . Q . . . .
. . . . . . . . Q . . .
. . . . . . . . . Q . .
. . . . . . . . . . Q .
. . . . . . . . . . . Q

```

3. Water Jug Puzzle (11L and 9L → Get 8L):

CODE:from collections import deque

cap1=11

cap2=9

goal=8

def water_jug():

 queue=deque([(0,0,[])])

 visited=set()

 while queue:

 a,b,path=queue.popleft()

 if (a,b) in visited:

 continue

 visited.add((a,b))

 if a==goal or b==goal:

 return path+[(a,b)]

 states=[

```

        (cap1,b,"Fill 11L"),
        (a,cap2,"Fill 9L"),
        (0,b,"Empty 11L"),
        (a,0,"Empty 9L"),

    ]
    pour=min(a,cap2-b)
    states.append((a-pour,b+pour,"Pour 11L to 9L"))

    pour=min(b,cap1-a)
    states.append((a+pour,b-pour,"Pour 9L to 11L"))

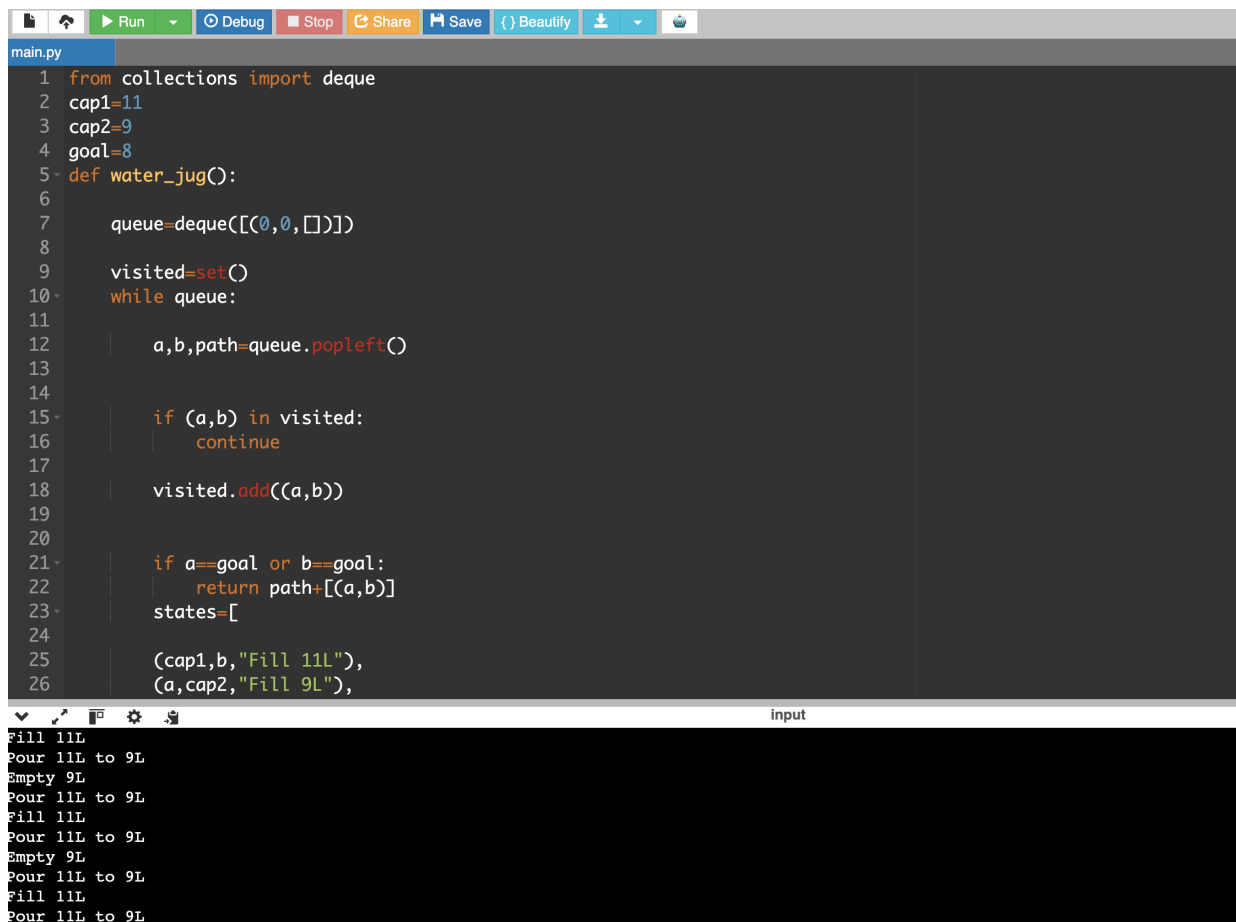
    for x,y,action in states:
        queue.append((x,y,path+[action]))
solution=water_jug()

```

for s in solution:

print(s)

OUTPUT:



The screenshot shows a code editor with a Python script for the water jug problem. The script defines a function `water_jug()` that uses a breadth-first search algorithm to find the shortest path to reach the goal state (2, 8). The script includes a queue, a visited set, and a list of states. The output of the script is displayed in a terminal window below the code editor.

```

main.py
1 from collections import deque
2 cap1=11
3 cap2=9
4 goal=8
5 def water_jug():
6
7     queue=deque([(0,0,[])])
8
9     visited=set()
10    while queue:
11
12        a,b,path=queue.popleft()
13
14
15        if (a,b) in visited:
16            continue
17
18        visited.add((a,b))
19
20
21        if a==goal or b==goal:
22            return path+[(a,b)]
23        states=[
24
25            (cap1,b,"Fill 11L"),
26            (a,cap2,"Fill 9L"),

```

Fill 11L
 Pour 11L to 9L
 Empty 9L
 Pour 11L to 9L
 Fill 11L
 Pour 11L to 9L
 Empty 9L
 Pour 11L to 9L
 Fill 11L
 Pour 11L to 9L

4. Tic-Tac-Toe AI using Minimax

```
CODE:board=[
```

```
' ',' ',' '
' ',' ',' '
' ',' ',' '
']
```

```
def show():
```

```
    print(board[0:3])
```

```
    print(board[3:6])
```

```
    print(board[6:9])
```

```
def check():
```

```
    win=[
```

```
        (0,1,2),(3,4,5),(6,7,8),
```

```
        (0,3,6),(1,4,7),(2,5,8)
```

```
    ]
```

```
    for a,b,c in win:
```

```
        if board[a]==board[b]==board[c]!=' ':
```

```
            return board[a]
```

```
    return None
```

```
while True:
```

```
    show()
```

```
    pos=int(input("Enter position 0-8: "))
```

```
    board[pos]='X'
```

```
    if check():
```

```
        print("You Win")
```

```
        break
```

```
    for i in range(9):
```

```
        if board[i]==' ':
```

```
            board[i]='O'
```

```
            break
```

```
    if check():
```

```
        show()
```

```
        print("AI Wins")
```

```
        break
```

OUTPUT:

```
main.py
1- board=[
2-     ' ', ' ', ' ',
3-     ' ', ' ', ' ',
4-     ' ', ' ', ' '
5- ]
6-
7- def show():
8-
9-     print(board[0:3])
10-    print(board[3:6])
11-    print(board[6:9])
12-
13- def check():
14-
15-     win=[
16-         (0,1,2),(3,4,5),(6,7,8),
17-         (0,3,6),(1,4,7),(2,5,8)
18-     ]
19-
20-     for a,b,c in win:
21-
22-         if board[a]==board[b]==board[c]!=' ':
23-             return board[a]
24-
25-     return None
26- while True:
```

```
input
['O', 'O', ' ']
['X', 'X', ' ']
[' ', ' ', ' ']
Enter position 0-8: 2
['O', 'O', 'X']
['X', 'X', 'O']
[' ', ' ', ' ']
Enter position 0-8: 5
You Win
```

5. 8-Puzzle using A* Algorithm:

CODE:import heapq

goal=(1,2,3,

4,5,6,

7,8,0)

start=(1,2,3,

4,0,6,

7,5,8)

def heuristic(state):

count=0

for i in range(9):

if state[i]!=0 and state[i]!=goal[i]:

count+=1

return count

def solve():

pq=[]

heapq.heappush(

pq,(0,start,[])

)

```

visited=set()
while pq:
    cost,state,path=heapq.heappop(pq)
    if state==goal:
        return path+[state]
    if state in visited:
        continue
    visited.add(state)
    zero=state.index(0)
    moves=[-1,1,-3,3]
    for m in moves:
        new=zero+m
        if 0<=new<9:
            s=list(state)
            s[zero],s[new]=s[new],s[zero]
            s=tuple(s)
            heapq.heappush(
                pq,
                (cost+1+heuristic(s),
                 s,
                 path+[state])
            )
solution=solve()
for s in solution:
    print(s)

```

OUTPUT:

```
main.py
1 import heapq
2
3 goal=(1,2,3,
4       4,5,6,
5       7,8,0)
6 start=(1,2,3,
7        4,0,6,
8        7,5,8)
9
10 def heuristic(state):
11     count=0
12
13     for i in range(9):
14         if state[i]!=0 and state[i]!=goal[i]:
15             count+=1
16
17     return count
18
19 def solve():
20
21     pq=[]
22
23     heapq.heappush(
24         pq,(0,start,[]))
25
26
input
(1, 2, 3, 4, 0, 6, 7, 5, 8)
(1, 2, 3, 4, 5, 6, 7, 0, 8)
(1, 2, 3, 4, 5, 6, 7, 8, 0)
...Program finished with exit code 0
Press ENTER to exit console.
```